Republic of the Philippines
CAMARINES SUR POLYTECHNIC COLLEGES
Nabua, Camarines Sur

COLLEGE *of* COMPUTER STUDIES

CCCS 104 - Data Structures and Algorithms

LEARNING TASK (LINEAR DATA STRUCTURE - STACK)

NAME: CARL RENZ M. COLICO SECTION: BSCS 2-A

RATIONALE

The given Learning Task presents an opportunity to reinforce the concepts of stack array, which are fundamental to data structures. A **stack** is a linear data structure that follows the **Last-In-First-Out (LIFO) principle**. This means that the last element added to the stack will be the first one to be removed. Think of it like a stack of plates: you **add plates to the top and also remove them from the top**. Some of the most common examples of Stack linear data structures include(s):

Browser History: The most recent page visited is the first to be removed when you go back.

Undo Mechanism: The last action performed is the first to be undone.

Expression Evaluation: Used in parsing expressions and evaluating postfix expressions.

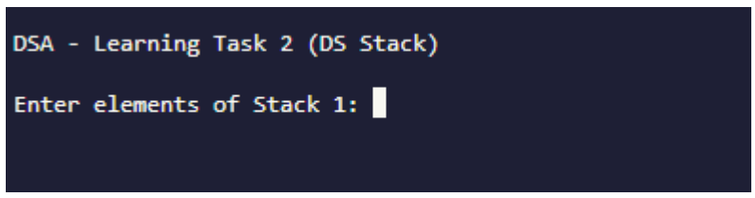
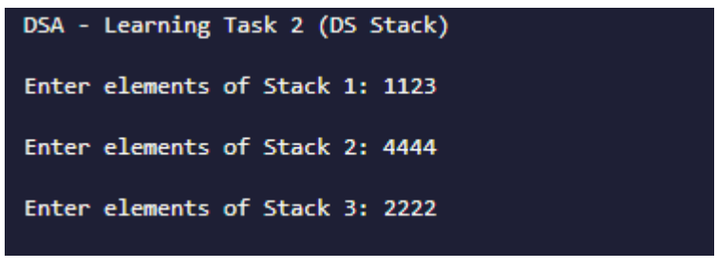
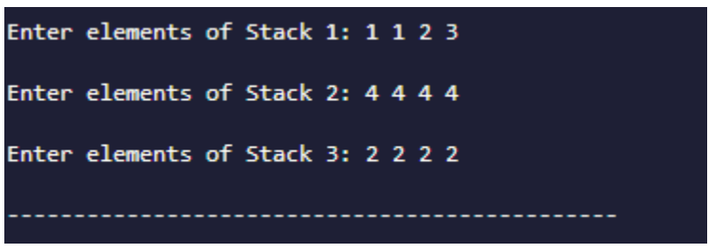
Depth-First Search (DFS): Used in graph traversal algorithms.

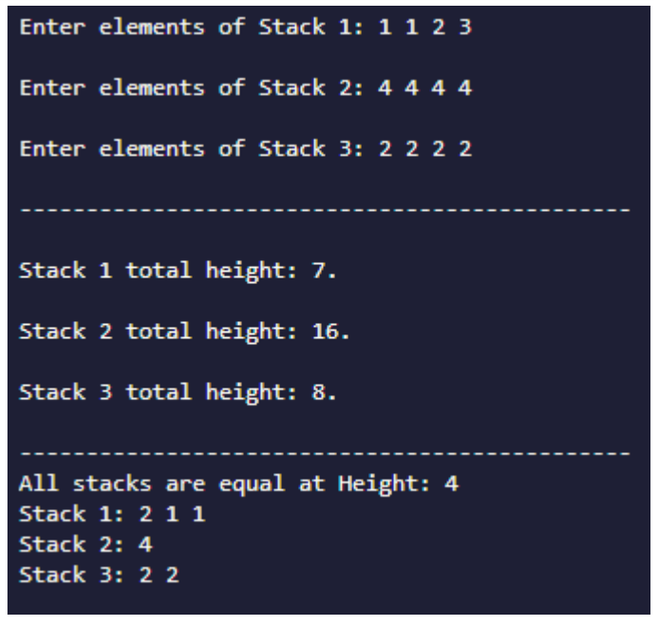
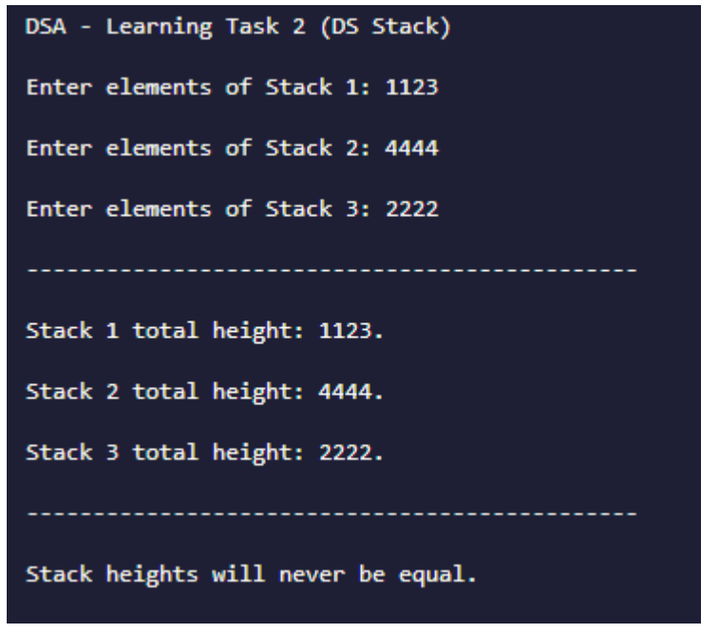
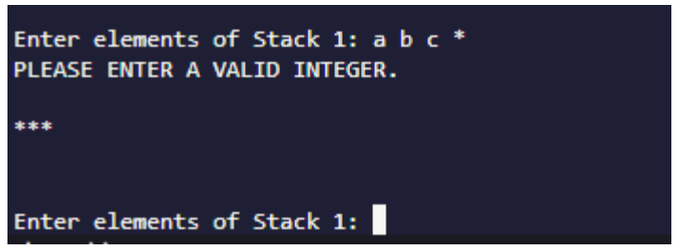
Linear data structures called Stacks offer various methods and functions, such as **Push to insert elements into the stack** and **Pop to eliminate the top element from the stack**. Stacks are essential for handling function calls, evaluating expressions, and implementing undo/redo features. In this learning task, the objective is to develop a stack array. Unlike a linked list that stores data in a series of connected nodes or chains, a stack array adds the last element to the stack and removes the first one.

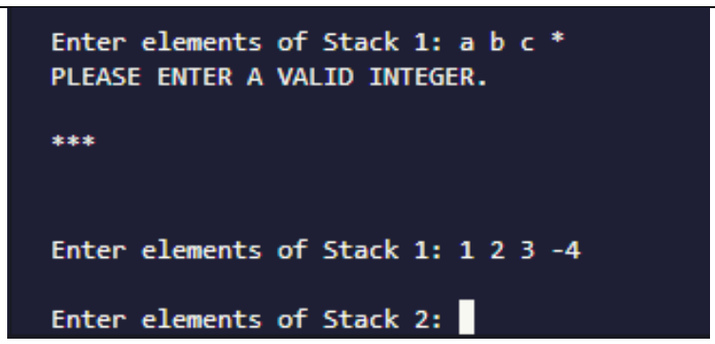
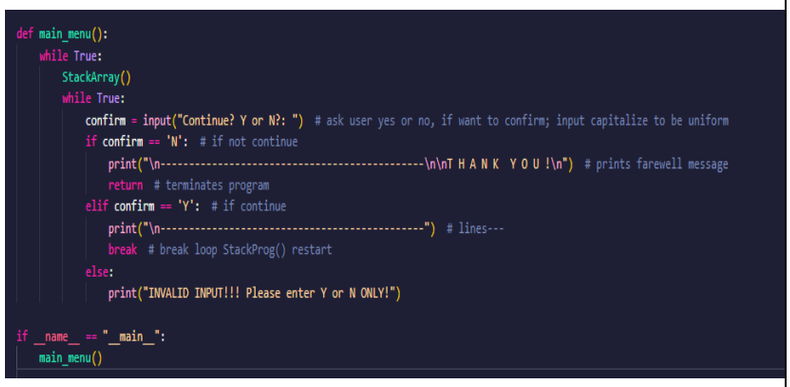
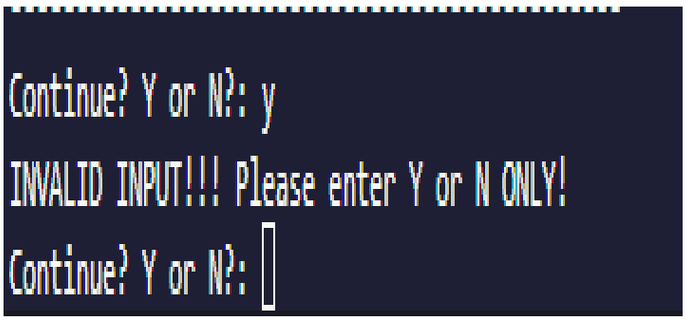
For this learning task, the assignment is to create a Python program that implements an array-based data structure based on the given requirements to implement a Stack Abstract Data Structure using an Array. The program should determine the height of each stack and find out at which height the three stacks are equal. The user will be asked to input the elements of each stack, with the inputs separated by spaces. Once the Enter key is pressed, the next prompt will be displayed accordingly. If the stacks are not equal at a given height, a message prompt will appear: "Stack heights will never be equal." Success and error messages should be displayed when necessary. The program will continue to loop until the user chooses not to continue, or N is pressed.

USER GUIDE

Step by step instructions on how to use your program. Include images for easily visualization

Step 1 *The prompting of running the terminal of code will ask the user to 'Enter elements of Stack [n]:*	*Image 1* 
Step 2 *Please remember the following instructions: When entering the elements of each number into the stack, the user should input the integers separately, using spaces. If the user fails to use a space before each integer or number they desire, the stack will read it as a single number rather than as separate numbers.*	 *WRONG METHOD*  *RIGHT METHOD*

Step 3 *The prompting method of Stack total height has two parts, the one that has a equal number of height and non-equal height. To state or print the equal height of all stack make sure the input elements in 'Enter element of Stack [n]: equal to each other or has the same totality of elements.* *But if the user doesn't want it to be equal then the user may input in 'Enter elements of Stack [n] : non-equal to each other or has the non-equal totality of elements in their desired stacks.*	 *EQUAL HEIGHT*  *NON - EQUAL HEIGHT*
Step 4 *The entering of elements of the stack only accepts **'integers'** and does not accept 'strings', 'letters', 'special symbol', entering such keys like this will result to an error. This will make the user to return*	

and prompt again to make enter the right element for the stack but, if the user again did not correctly input right integers then it will return again the 'Enter elements of Stack [n]:' ***Until the user make a right input. Then that is the time the user can input to another count of stack.***	
Step 5 *--EXIT MAIN-MENU—* *If the user wants to exit the program, they can choose the Letter [N] Exit operation and it will prompt '[THANK YOU]. If they proceed to enter [Y] it will 'RETURN' and continue the program.* *Note: THE EXIT METHOD ONLY ACCEPTS BIG OR UPPER CASE LETTER '[Y]' and '[N]'. If the user inputted a wrong valid exception then it will process the error prompt and 'RETURN' to ask again the user to enter UPPER CASE LETTER '[Y]' and '[N]'.*	 

PROGRAM CODE

```
print("\nDSA - Learning Task 2 (DS Stack)")

def push(stack, item): #defining the pushing method with it parameter of stack
and a item/value in our stack, to let us input item in our stackarr
    stack.append(item) # then this will append or just insert that item in a
first in last out method


def pop(stack): #defining the popping method or the deletion with its parameter
of a stack as what this just do is delete the item in our stack_arr
    if len(stack) == 0:
        #CASE 0 : Empty array
        return None
    return stack.pop()


def StackArray(position=None):
    empty_stacks = []
    empty_heights = []
    stack_count = 3  # the number of the stack status is set to three

    for data in range(stack_count):  # the parameter to argument our 3 stacks
and has an array of items:
        while True:  # CASE 1 : Array has items of elements that is not int
            try:  # exception handling
                integer = input(f'\nEnter elements of Stack {data + 1}:
').split()
                integer = [int(num) for num in integer]
                break
            except ValueError:
                print("PLEASE ENTER A VALID INTEGER. \n\n***\n")

        stacks = []  # initialize a set of stack that is not empty
        heights = 0  # initialize a set of height that is not empty

        for values in integer:
            push(stacks, values)
            heights += values

        empty_stacks.append(stacks)
        empty_heights.append(heights)

    print("\n---------------------------------------------- ")

    for i in range(len(empty_heights)):  #  HEIGHT IS NOT EMPTY AND HAS A
VALUE.
        print(f'\nStack {i + 1} total height: {empty_heights[i]}.')

    print("\n---------------------------------------------- ")

    # CASE 3: ELEMENTS OF EACH STACKS HEIGHT WILL BE NON-EQUAL.
```

```
    while len(set(empty_heights)) != 1:  # setting a while loop to check the
data elements of heights in our stacks
        if any(h == 0 for h in empty_heights):  # when node is empty before or
on specified position
            print("\nStack heights will never be equal.")
            print("\n------------------------------------------- ")
            return

        # CASE 4: USER INPUT A NEGATIVE ELEMENTS/INTEGERS.
        if all(h < 0 for h in empty_heights):
            for data in range(stack_count):
                if empty_heights[data] == min(empty_heights):  # for elements
in stackarray check and find the minimum height
                    poppin = pop(empty_stacks[data]) # typically removes and
returns the top element of stack and stored it to poppin .
                    if poppin is not None:  # exception: check if the array is
empty to none/ not empty.
                        empty_heights[data] -= poppin  # let us decrement its
value and reassign it with pop value.
                    break

        # CASE 5: This time for height that count the maximum height in our
created data or stacks
        for data in range(stack_count):
            if empty_heights[data] == max(empty_heights):  # check the
highest(maximum) heights in the three stack array
                poppin = pop(empty_stacks[data])# The pop function removes and
returns the top element of the stack.
                if poppin is not None: #If poppin is not None, it subtracts the
popped value
                     # exception: check if the array is empty to none/ not
empty.
                    empty_heights[data] -= poppin
                    break

        # CASE 6: THIS TIME the else method is for the initialization of
maximum heights in our data array stack
        else:
            for data in range(stack_count):
                if empty_heights[data] == max(empty_heights):  # for elements
in stackarray check and find the maximum height
                    poppin = pop(empty_stacks[data])
                    if poppin is not None:  # exception: check if the array is
empty to none/ not empty.
                        empty_heights[data] -= poppin
                        break

    # CASE 7: Chects all the elements in empty_heights if or are the same
heights or elements thus if is ==1
    # remember that the elements in our stack is in a set of datas, which will
haveonly 1 elem. making its length combined to 1
    if len(set(empty_heights)) == 1:
```

```
        print(f'All stacks are equal at Height: {empty_heights[0]}')  #  equal
height success, prints success message
    #for loop implement iterates over each stack in the empty_stacks list.
        for stackarr in range(len(empty_stacks)):  #It let the elements in our
stack array print
            print(f'Stack {stackarr + 1}: {" ".join(map(str,
empty_stacks[stackarr][::-1]))}')
            # so in its printing state it we used a .join method to join the
string main output and a map class is use to converts each element in the stack
to a string.

        print("\n-------------------------------------------- ")

def main_menu():
    while True:
        StackArray()
        while True:
            confirm = input("Continue? Y or N?: ")  # ask user yes or no, if
want to confirm; input capitalize to be uniform
            if confirm == 'N':  # if not continue
                print("\n-----------------------------------------------\n\nT H
A N K  Y O U !\n")  # prints farewell message
                return  # terminates program
            elif confirm == 'Y':  # if continue
                print("\n-----------------------------------------------")  #
lines---
                break  # break loop StackProg() restart
            else:
                print("INVALID INPUT!!! Please enter Y or N ONLY!")

if __name__ == "__main__":
    main_menu()
```

TUTORIAL VIDEO

YouTube Link: https://www.youtube.com/watch?v=7eWWq6q3GYs

TAKEAWAYS

My learnings to this given data structure using a stack array is I did learned such…

- Application of Pop(), Append(): for deletion and insertion method which is simple to built and add in the code structure.
- The used and importance of increment in adding (data +1) in the printing state to add the number count of stack array.
- The used and importance of decrement (data -=) in checking the value and reassign it with the pop method.
- The importance of using types of statement in handling exception (the if –else –while statement +.int() method in catching the errors so that the loop will no continue (if none) and terminate the loop also the while if it catches that the data has a only one element or consist of many elements and lastly the int to check whether it is a valid integer.
- Encountered Errors in looping structure implementations, such as not accepting zero(0) as integer,negative integers and the arrangement of print statements, but in the end I found ways with the help of pointers , references in the internet it help to debug the code.
- Converting visualizations of processes on how stack array Push which is to insert,Pop which is to delete, as well as set of stack array work into a real-dynamic code.